

ECCV 2026 Tutorial: Efficient MLLM Inference via Approximate and Exact Computing

Approximate Computing: Model Compression

Sathya N. Ravi
University of Illinois Chicago
09/08/2026

Introduction

The MLLM Explosion: Capabilities vs. Scale

State-of-the-Art (2024–2026)

- **Proprietary:** GPT-4o, Gemini 1.5 Pro, Claude 3.5 (Millions of context tokens)
- **Open Weights:** Llama-3-Vision, LLaVA-NeXT, Qwen-VL (70B+ parameters)
- **The Trend:** Multimodal Leaderboards (OpenCompass, LMSYS) are completely dominated by models requiring multi-GPU setups.

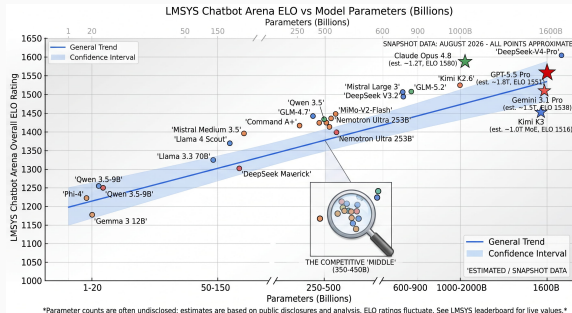


Figure 1: Model Performance vs. Parameter Count, source: [link](#)

The Harsh Reality: The Inference Bottleneck

Memory is the New Compute

- A 70B parameter model in FP16 requires **140GB** of VRAM just to store weights.
- *KV Cache* for high-res images and long video sequences scales linearly with context length.
- **Result:** Inference is heavily bottlenecked by memory bandwidth, not just compute FLOPS.

Industry Headlines

"AI infrastructure power demands threaten grid limits.", "Inference costs outpace training costs in deployment."

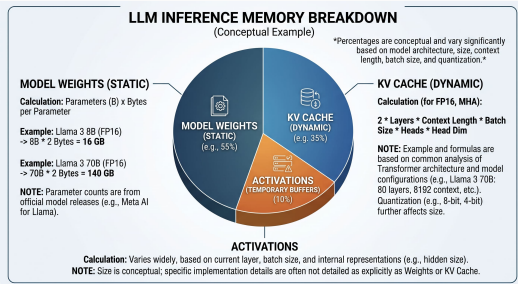


Figure 2: VRAM Allocation Breakdown (LLM)

Why Model Compression?

- **Environmental & Hardware Limits:** Training is a one-time cost; inference is perpetual. Data center GPU availability remains constrained.
- **Edge Deployment:** Autonomous driving, robotics, and medical AI (e.g., real-time echocardiogram analysis) require low-latency, on-device vision-language reasoning.
- **Democratization:** Enabling 70B+ MLLMs on single consumer GPUs or local edge servers.

The Core Objective for Approximate Computing

How do we shrink a massive MLLM to fit within severe hardware constraints while retaining 99% of its (zero-shot) visual reasoning capabilities?

Roadmap: Approximate Computing for MLLMs



- **Part I:** Weight and Activation Quantization (W4A8, AWQ, GPTQ)
- **Part II:** Pruning Vision Encoders (ViT) and LLM Backbones
- **Part III:** Token Merging and Efficient Attention for Visual Context

Model Quantization: Mathematical Foundations & Algorithms

The Quantization Mapping: Basics

Core Idea: Map continuous floating-point values (e.g., FP16) to a discrete set of lower-precision integers (e.g., INT8, INT4).

Given a tensor $\mathbf{x} \in \mathbb{R}^N$, the standard affine (asymmetric) quantization function is:

$$Q(\mathbf{x}) = \text{clip} \left(\left\lfloor \frac{\mathbf{x}}{S} \right\rfloor + Z, \quad 0, \quad 2^b - 1 \right) \quad (1)$$

where:

- b is the bit-width (e.g., 4 or 8).
- $S \in \mathbb{R}^+$ is the **scale factor**.
- $Z \in \mathbb{Z}$ is the **zero-point** (ensures real 0.0 is exactly representable).

Dequantization (approximating the original tensor):

$$\hat{\mathbf{x}} = S \cdot (Q(\mathbf{x}) - Z) \approx \mathbf{x} \quad (2)$$

Quantization Granularity

The choice of how to group parameters to share the scale S and zero-point Z dictates the trade-off between memory and accuracy.

- **Per-Tensor:** One (S, Z) for the entire weight matrix. (Fastest, lowest memory, highest error).
- **Per-Channel/Column:** One (S, Z) per output channel. (Standard for W8A8).
- **Per-Group:** Weights are grouped into blocks of size G (e.g., $G = 64$ or 128) along the input channel. One (S, Z) per group. (Standard for W4A16).

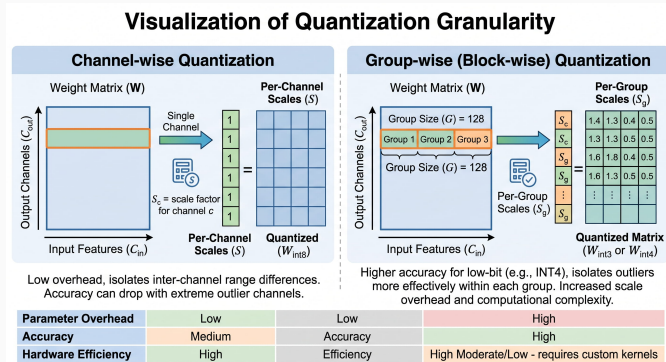


Figure 3: Visualizing Group-wise vs Channel-wise Quantization.

The LLM/MLLM Quantization Challenge

Why can't we just use **naive** Post-Training Quantization (PTQ)?

Naive PTQ (Min-Max rounding) worked fine for ResNets and BERT, but fails catastrophically for LLMs.

The Activation Outlier Phenomenon

As models scale beyond 6B parameters, massive **systematic outliers** emerge in the activation tensors (but *not* in the weights).

- Outliers can be $100\times$ larger than normal activations.
- They concentrate heavily in a small subset of specific channels.
- If we apply naive per-tensor quantization, these outliers force the scale factor S to be massive, collapsing 99% of the useful signal into a single quantization bin.

Paradigm 1: Weight-Only Quantization (W4A16)

During the decoding phase (auto-regressive generation), MLLM inference is **memory-bandwidth bound**.

- **Idea:** Only quantize the weights (to INT4) to save VRAM and memory bandwidth. Keep activations in FP16.
- **Execution:** Weights are loaded from HBM to SRAM in INT4, dequantized to FP16 in registers on-the-fly, and multiplied with FP16 activations.

Popular Algorithms:

1. **GPTQ:** Second-order (Hessian) based weight adjustment.
2. **AWQ:** Activation-aware scaling of salient weights.

GPTQ (Frantar et al., 2022) formulates weight quantization as a layer-wise optimal reconstruction problem.

Let W be the original weights, X be a small calibration set of inputs, and $\hat{W}$ be the quantized weights. We want to minimize the squared error of the output:

$$\arg \min_{\hat{W}} \|WX - \hat{W}X\|_F^2 \quad (3)$$

We will see that the optimal update for the **remaining** unquantized weights (when one weight is quantized) depends on the inverse Hessian matrix $\mathbf{H}^{-1}$, where $\mathbf{H} = 2XX^T$.

GPTQ: Decomposable Objective

Dimensions in a single linear layer:

- $W, \hat{W} \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$
- $X \in \mathbb{R}^{d_{\text{in}} \times N}$ (N = tokens in calibration set)

The global objective uses the squared Frobenius norm, which decouples into independent problems for each row $\mathbf{w} \in \mathbb{R}^{1 \times d_{\text{in}}}$ (or for each column):

$$\arg \min_{\hat{W}} \|WX - \hat{W}X\|_F^2 \implies \sum_{i=1}^{d_{\text{out}}} \arg \min_{\hat{\mathbf{w}}_i} \|\mathbf{w}_i X - \hat{\mathbf{w}}_i X\|_2^2 \quad (4)$$

Expanding the row-wise L_2 norm yields a quadratic formulation:

$$(\mathbf{w} - \hat{\mathbf{w}})XX^T(\mathbf{w} - \hat{\mathbf{w}})^T = \frac{1}{2}(\mathbf{w} - \hat{\mathbf{w}})\mathbf{H}(\mathbf{w} - \hat{\mathbf{w}})^T \quad (5)$$

where the Hessian is $\mathbf{H} = 2XX^T$.

GPTQ: First-Order Optimality Conditions

Quantizing the q -th weight to a fixed scalar c introduces an equality constraint:

$$\mathbf{e}_q^T \hat{\mathbf{w}} = c.$$

We form the Lagrangian to find the optimal adjustment for all remaining unquantized weights:

$$\mathcal{L}(\hat{\mathbf{w}}, \lambda) = \frac{1}{2}(\mathbf{w} - \hat{\mathbf{w}})\mathbf{H}(\mathbf{w} - \hat{\mathbf{w}})^T - \lambda(\mathbf{e}_q^T \hat{\mathbf{w}} - c) \quad (6)$$

Setting the gradient to zero provides the optimality condition:

$$\nabla_{\hat{\mathbf{w}}} \mathcal{L} = -\mathbf{H}(\mathbf{w} - \hat{\mathbf{w}})^T - \lambda \mathbf{e}_q = 0 \implies (\mathbf{w} - \hat{\mathbf{w}})^T = -\lambda \mathbf{H}^{-1} \mathbf{e}_q \quad (7)$$

Solving for the dual variable λ via the constraint gives $\lambda = -\frac{\delta}{[\mathbf{H}^{-1}]_{qq}}$, where $\delta = w_q - c$ is the quantization error.

GPTQ: The Final Update Rule

Substituting λ back into the first-order condition yields the exact, optimal update for the continuous weights in $\hat{\mathbf{w}}$ to compensate for quantizing w_q :

Optimal Brain Quantization (OBQ) Step

$$\mathbf{w} \leftarrow \mathbf{w} - \frac{\delta}{[\mathbf{H}^{-1}]_{qq}} \cdot (\mathbf{H}^{-1})_{:,q} \quad (8)$$

Implementation constraints: Instead of recomputing the inverse Hessian for every single parameter, GPTQ quantizes columns in blocks and uses a Cholesky decomposition to update $\mathbf{H}^{-1}$ efficiently, scaling this exact solver to 70B+ parameter MLLMs.

AWQ: Activation-aware Weight Quantization

AWQ (Lin et al., 2023) argues that we don't need expensive Hessian tracking. Not all weights are equally important. Weights corresponding to *large activation magnitudes* (salient weights) dictate the output error.

Core Insight: Preserving just $\sim 1\%$ of salient weights in FP16 significantly reduces quantization error. However, mixed precision is slow on hardware.

Solution: Mathematically scale the salient weights up, and scale their corresponding activations down!

AWQ: Optimization Formulation

Unlike GPTQ which compensates via the inverse Hessian, AWQ frames quantization as finding an optimal channel-wise scaling vector $\mathbf{s} \in \mathbb{R}^{d_{\text{in}}}$.

For a linear layer $Y = WX$, applying a diagonal scaling matrix $S = \text{diag}(\mathbf{s})$ yields the mathematically equivalent transformation:

$$Y = (WS)(S^{-1}X) \quad (9)$$

The global objective minimizes the reconstruction error of the output feature map under the quantization operator $Q(\cdot)$:

$$\mathbf{s}^* = \arg \min_{\mathbf{s}} \|WX - Q(W\text{diag}(\mathbf{s}))(\text{diag}(\mathbf{s})^{-1}X)\|_F^2 \quad (10)$$

The Bottleneck: This objective is highly non-convex due to the discrete nature of $Q(\cdot)$. Gradient-based optimization (e.g., using a Straight-Through Estimator) frequently falls into poor local minima and severely overfits the limited calibration data.

AWQ: The Tractable Search Space

To make the non-convex optimization tractable and robust, AWQ heavily restricts the search space of $\mathbf{s}$ by tying it directly to the activation statistics.

Let $\mathbf{m}_X \in \mathbb{R}^{d_{\text{in}}}$ be the channel-wise mean absolute magnitude of the calibration activations. The scaling vector is parameterized by a single scaling exponent $\alpha \in [0, 1]$:

$$\mathbf{s}(\alpha) = \mathbf{m}_X^\alpha \quad (11)$$

- $\alpha = 0 \implies \mathbf{s} = \mathbf{1}$ (Recovers naive PTQ; no scaling).
- $\alpha = 1 \implies \mathbf{s} = \mathbf{m}_X$ (Full activation-aware scaling).

The Solver: By collapsing the d_{in} -dimensional search space into a 1-dimensional manifold per layer, the optimal α is found via a rapid, discrete 1D grid search over $[0, 1]$, making the algorithm extremely fast and completely avoiding backpropagation.

Paradigm 2: W8A8 Quantization (SmoothQuant)

For the **Prefill phase** (processing thousands of visual and text tokens at once), inference is **compute-bound**. W4A16 doesn't speed up dense matrix multiplication (GEMM), so we need to quantize activations also.

SmoothQuant (Xiao et al., 2023) solves the activation outlier problem to enable W8A8.

$$Y = (X \text{diag}(\mathbf{s})^{-1}) \cdot (\text{diag}(\mathbf{s})W) = \hat{X} \hat{W} \quad (12)$$

SmoothQuant "smooths" the activations by migrating the variance (the outliers) from the activations X into the weights W .

SmoothQuant: Migration Factor α

How do we balance the difficulty of quantizing weights vs. activations?

SmoothQuant introduces a migration factor $\alpha \in [0, 1]$:

$$s_j = \max(|X_j|)^\alpha / \max(|W_j|)^{1-\alpha} \quad (13)$$

- $\alpha = 1$: Push all difficulty to weights (makes weights impossible to quantize).
- $\alpha = 0$: Push all difficulty to activations (original problem).
- $\alpha = 0.5$: Equalizes the max values of channels in weights and activations.

Typically optimal for LLMs.

The Multimodal Bottleneck: Why LLM PTQ Fails on MLLMs

Standard techniques (SmoothQuant/AWQ) applied naively to MLLMs (like LLaVA) often degrade visual reasoning severely.

The Vision-Text Disparity

Visual tokens (from ViT/CLIP) and Text tokens have fundamentally different activation distributions.

- Visual token activations can be $10\times$ to $20\times$ larger than text tokens.
- A unified global scaling factor s over penalizes text tokens, causing "smoothing misalignment."

MASQuant: Modality-Aware Smoothing

Recent works (e.g., LLaVA-PTQ, MASQuant) propose treating visual and textual modalities separately during the smoothing phase.

$$Y = [X_{\text{vis}}, X_{\text{txt}}]W = [\hat{X}_{\text{vis}}\mathbf{s}_{\text{vis}}^{-1}, \hat{X}_{\text{txt}}\mathbf{s}_{\text{txt}}^{-1}]W \quad (14)$$

- Learn separate channel-wise scaling vectors for $\mathbf{s}_{\text{vis}}$ and $\mathbf{s}_{\text{txt}}$.
- Apply **Cross-Modal Compensation (CMC)**: since the weight matrix W is fixed, dynamic shifting between text and vision scales requires low-rank compensation branches during inference to maintain mathematical equivalence.

The Third Paradigm: KV Cache Quantization

The Long-Context Memory Wall

While GPTQ and AWQ compress the model's **weights**, they ignore the memory required to store the generation history: the **KV Cache**.

The KV Cache Bottleneck

During autoregressive decoding, the Key (K) and Value (V) tensors for all previous tokens must be kept in VRAM.

- Scales linearly: $\mathcal{O}(B \times S \times L \times H \times D)$
(Batch $\times$ Sequence Length $\times$ Layers $\times$ Heads $\times$ Dim)
- For a 100K token visual context (e.g., video processing) on a 70B model, the KV cache can easily exceed **100 GB** of VRAM alone.

So, quantizing the KV cache to 4 bits or lower is preferred, but naive PTQ is not robust to activation outliers.

TurboQuant: The Outlier Smoothing Trick

TurboQuant (ICLR, 2026) tackles the outlier problem in KV caches without sorting or using variable bit-rates (which GPUs hate).

The Core Idea: Random Rotation Instead of directly quantizing a spiky, KV vector with outliers $\mathbf{x}$, TurboQuant multiplies it by a random orthogonal matrix R (often implemented via a Randomized Hadamard Transform for speed).

$$\mathbf{x}_{\text{rot}} = R\mathbf{x} \quad (15)$$

Why? The random rotation smears the massive outliers evenly across all dimensions. So, every coordinate shares roughly the same scale!

TurboQuant-MSE: Mathematical Formulation

Once rotated, by the concentration of measure in high dimensions, the coordinates of $\mathbf{x}_{\text{rot}}$ follow a shifted and scaled **Beta distribution** (which converges to a Gaussian).

Because the coordinates are now identically distributed and near-independent, TurboQuant applies an optimal 1D scalar quantizer independently to each dimension:

$$\arg \min_{\hat{\mathbf{x}}} \|\mathbf{x}_{\text{rot}} - \hat{\mathbf{x}}_{\text{rot}}\|_2^2 \quad (16)$$

The optimal quantization bins (centroids) are found continuously using the **Lloyd-Max algorithm** (1D K-means) based on the theoretical Beta distribution, making the algorithm completely data-oblivious (no calibration set required!).

The Inner Product Problem in Attention

Why does minimizing MSE destroy the Attention mechanism?

Attention scores dictate token probabilities via the softmax function:

$$A_{ij} = \text{softmax} \left(\frac{\mathbf{q}_i^T \mathbf{k}_j}{\sqrt{d}} \right) \quad (17)$$

Let the quantized key be $\hat{\mathbf{k}}_j$ with residual $\mathbf{r}_j = \mathbf{k}_j - \hat{\mathbf{k}}_j$. The estimated attention score is:

$$\mathbf{q}_i^T \mathbf{k}_j = \mathbf{q}_i^T \hat{\mathbf{k}}_j + \mathbf{q}_i^T \mathbf{r}_j \quad (18)$$

The Problem

Standard quantization (Lloyd-Max) minimizes the L_2 norm of the residual, $\|\mathbf{r}_j\|_2^2$.

However, in high dimensions, a vector with a tiny norm can still drastically alter the *direction* of the key. If $\mathbf{r}_j$ happens to align with the query $\mathbf{q}_i$, the dot product $\mathbf{q}_i^T \mathbf{r}_j$ blows up, the softmax distribution collapses, and the LLM hallucinates.

Quantized Johnson-Lindenstrauss (QJL): Setup

To fix this, TurboQuant-Prod explicitly estimates the missing dot product $\mathbf{q}^T \mathbf{r}$ using just **1 bit** per dimension via the Quantized Johnson-Lindenstrauss transform.

Let $W \in \mathbb{R}^{m \times d}$ be a random projection matrix with i.i.d. entries $W_{uv} \sim \mathcal{N}(0, 1)$.

Encoding the Key Residual (Cached once): During prefill, we project the residual $\mathbf{r}$ and store only its sign bits:

$$\mathbf{z} = \text{sign}(W\mathbf{r}) \in \{-1, +1\}^m \quad (19)$$

Cost: Exactly 1 bit per dimension, plus a single FP16 scalar for the norm $\|\mathbf{r}\|_2$.

Projecting the Query (Computed on-the-fly): During decoding, the dense query is projected using the same seed for W :

$$\mathbf{y} = W\mathbf{q} \in \mathbb{R}^m \quad (20)$$

QJL: The Unbiased Expectation Proof

How does a 1-bit sign vector recover an exact continuous dot product? Consider the k -th coordinate. We want to find $\mathbb{E}[y_k z_k] = \mathbb{E}[(\mathbf{w}_k^T \mathbf{q}) \text{sign}(\mathbf{w}_k^T \mathbf{r})]$.

Decompose $\mathbf{q}$ into components parallel and orthogonal to $\mathbf{r}$:

$$\mathbf{q} = \frac{\mathbf{q}^T \mathbf{r}}{\|\mathbf{r}\|_2^2} \mathbf{r} + \mathbf{q}^\perp \quad (21)$$

Substituting this into the query projection:

$$\mathbf{w}_k^T \mathbf{q} = \frac{\mathbf{q}^T \mathbf{r}}{\|\mathbf{r}\|_2^2} (\mathbf{w}_k^T \mathbf{r}) + \mathbf{w}_k^T \mathbf{q}^\perp \quad (22)$$

Because $\mathbf{w}_k$ is an isotropic Gaussian, $\mathbf{w}_k^T \mathbf{r}$ and $\mathbf{w}_k^T \mathbf{q}^\perp$ are independent zero-mean Gaussians. Thus, the cross-term expectation $\mathbb{E}[(\mathbf{w}_k^T \mathbf{q}^\perp) \text{sign}(\mathbf{w}_k^T \mathbf{r})] = 0$.

QJL: The Unbiased Expectation Proof (Cont.)

We are left with the parallel component:

$$\mathbb{E}[y_k z_k] = \frac{\mathbf{q}^T \mathbf{r}}{\|\mathbf{r}\|_2^2} \mathbb{E} \left[(\mathbf{w}_k^T \mathbf{r}) \text{sign}(\mathbf{w}_k^T \mathbf{r}) \right] \quad (23)$$

Notice that $X = \mathbf{w}_k^T \mathbf{r}$ is a Gaussian random variable $X \sim \mathcal{N}(0, \|\mathbf{r}\|_2^2)$. By definition, $X \text{sign}(X) = |X|$. The expected absolute value of a Gaussian is:

$$\mathbb{E}[|X|] = \sqrt{\frac{2}{\pi}} \|\mathbf{r}\|_2 \quad (24)$$

Substituting this back and summing over all m coordinates of the projection:

$$\mathbb{E}[\mathbf{y}^T \mathbf{z}] = m \sqrt{\frac{2}{\pi}} \frac{\mathbf{q}^T \mathbf{r}}{\|\mathbf{r}\|_2} \implies \mathbf{q}^T \mathbf{r} \approx \sqrt{\frac{\pi}{2}} \frac{\|\mathbf{r}\|_2}{m} (\mathbf{y}^T \mathbf{z}) \quad (25)$$

Putting it together for Attention: The full, unbiased attention score computation during decoding becomes:

$$\mathbf{q}_i^T \mathbf{k}_j \approx \mathbf{q}_i^T \hat{\mathbf{k}}_j + \left(\sqrt{\frac{\pi}{2}} \frac{\|\mathbf{r}_j\|_2}{m} \right) \mathbf{y}_i^T \mathbf{z}_j \quad (26)$$

Implementation Advantages

- $\mathbf{y}_i = W\mathbf{q}_i$ is computed *once* per generated token, not per KV cache element.
- $\mathbf{z}_j$ is loaded from VRAM as packed 1-bit integers.
- The dot product $\mathbf{y}_i^T \mathbf{z}_j$ maps directly to highly optimized hardware instructions (e.g., 'dp4a' or Tensor Core bitwise ops), avoiding FP16 memory bandwidth limits entirely.

TurboQuant-Similarity: Fixing the Attention Bias

The Cosine Similarity Problem

Minimizing Mean Squared Error (MSE) is great, but the Attention mechanism relies on inner products (cosine similarity). Rounding errors change the *direction* of the vectors, causing zero-shot intelligence to collapse.

The Solution (TurboQuant_{prod}): A Two-Stage Quantizer.

1. Use $b - 1$ bits for the TurboQuant-MSE step to minimize absolute distance.
2. Compute the residual error (the direction lost during rounding).
3. Use the final **1 bit** to encode a Quantized Johnson-Lindenstrauss (QJL) transform on the residual.

This 1-bit compass explicitly acts to unbiased the inner product estimation, ensuring the attention scores remain perfectly calibrated.

TurboQuant: Inference Results

Method	Bit-Width	Needle-in-a-Haystack	Effective Context (VRAM)
FP16 (Uncompressed)	16-bit	100%	1×
Naive Min-Max PTQ	4-bit	Fails (< 20%)	4×
TurboQuant	3.5-bit	99.8%	~ 4.5×
TurboQuant	2.5-bit	94.0%	~ 6.4×

Table 1: Performance retention on Long-Context MLLM tasks.

Takeaway: By combining Random Rotation with mathematically derived Lloyd-Max centroids, TurboQuant allows for 4× longer context windows on the exact same hardware without blowing up GPU memory or dropping the AI’s intelligence.

Summary: Quantization for MLLMs

Key Takeaways

1. **Memory is the bottleneck** in decoding; **Compute is the bottleneck** in context prefill.
2. **W4A16 (AWQ/GPTQ)**: Best for single-batch, long-generation tasks (Edge deployment).
3. **W8A8 (SmoothQuant)**: Best for high-throughput, massive batch-size scenarios (Cloud serving).
4. **Multimodal specific PTQ**: Essential to prevent vision-token outliers from destroying text-token precision.

Next, we will explore what happens when we want to structurally remove parameters via **Pruning**.

Model Pruning & Low-Rank Approximation

Introduction to Neural Network Pruning

Core Objective: Remove redundant parameters from a trained model to reduce memory footprint and compute FLOPs, without sacrificing zero-shot capabilities.

Unstructured Pruning

- Drops individual weights based on a criterion (e.g., magnitude).
- Results in sparse matrices (e.g., SparseGPT, Wanda).
- **Pros:** High sparsity ratios with possibly minimal accuracy drop.
- **Cons:** Hardware (GPUs) struggles to accelerate unstructured sparsity without specialized kernels.

Structured Pruning (Low-Rank)

- Removes entire structural components (attention heads, channels, or rank).
- Factors a large matrix $W \in \mathbb{R}^{m \times n}$ into two smaller ones $A \in \mathbb{R}^{m \times r}$ and $B \in \mathbb{R}^{r \times n}$.
- **Pros:** Immediate wall-clock speedup on any hardware.

Pruning as Continuous Optimization

Before diving into zero-shot post-training heuristics, we must ground pruning in its continuous optimization roots.

The standard approach during training or fine-tuning applies a sparsity-inducing regularization penalty $\mathcal{R}(W)$:

$$\min_W \mathcal{L}(WX, Y) + \lambda \mathcal{R}(W) \quad (27)$$

While ℓ_0 norm (counting non-zeros) is NP-hard, the standard convex relaxation uses the ℓ_1 norm: $\mathcal{R}(W) = \|W\|_1$.

Proximal Splitting Methods for Exact Sparsity

To achieve exact zeros and true sparsity, we split the objective and apply **Proximal Gradient Descent** (e.g., ISTA / FISTA).

We take a gradient step on the smooth loss $\mathcal{L}$, followed by the proximal operator for the ℓ_1 penalty:

$$W^{(t+1)} = \text{prox}_{\alpha\lambda\|\cdot\|_1} \left(W^{(t)} - \alpha \nabla \mathcal{L}(W^{(t)}) \right) \quad (28)$$

The proximal operator for the ℓ_1 norm evaluates to the analytically closed-form **Soft-Thresholding Operator**:

$$\mathcal{S}_\tau(w) = \text{sign}(w) \max(|w| - \tau, 0) \quad (29)$$

Unlike subgradient descent, soft-thresholding safely collapses weights within the $[-\tau, \tau]$ band directly to exactly zero, avoiding terminal oscillation.

The Shift to Post-Training Pruning

While proximal methods are well behaved during training, MLLMs (70B+ parameters) are too expensive to retrain. We need zero-shot Post-Training Pruning.

Magnitude Pruning (The Baseline): Simply remove weights with the smallest absolute values:

$$\text{mask}_{ij} = \mathbb{I}(|W_{ij}| > \tau) \quad (30)$$

Failure mode in LLMs: As in Quantization, outliers in activations affect the network's prediction. A weight might be big, but if it multiplies a activation outlier, keeping it is not a great idea.

Wanda: Pruning by Weight and Activation

Wanda (Sun et al., 2023) solves this by incorporating activation statistics without requiring any gradient computation or inverse Hessian (unlike SparseGPT).

The pruning metric S_{ij} for a weight W_{ij} is the product of its magnitude and the L_2 norm of its corresponding input activation feature across a calibration set:

$$S_{ij} = |W_{ij}| \cdot \|X_j\|_2 \quad (31)$$

- $X_j \in \mathbb{R}^N$ is the vector of activations for the j -th input channel.
- Wanda prunes weights where S_{ij} is smallest.
- **Intuition:** S act as Effective weights so that small weights big activations and big weights small activations can both be pruned.

Structured Pruning in the Vision Encoder (ViT)

In an MLLM, the Vision Transformer (e.g., CLIP-ViT-L) processes thousands of high-resolution image patches.

Unlike the LLM backbone, the Vision Encoder exhibits high redundancy across **Attention Heads**.

Taylor Expansion for Head Pruning

To prune an entire attention head h , we can estimate the impact on the loss using a first-order Taylor expansion around the mask variable $z_h \in \{0, 1\}$.

We can prune heads with the lowest importance score $\mathcal{I}(h)$, drastically reducing the KV cache footprint required for the visual context.

Estimating Head Importance: The Setup

Goal: Quantify the impact of removing a specific attention head h on the global loss $\mathcal{L}$ without re-evaluating the entire network.

The output of a Multi-Head Self-Attention (MHSA) layer can be viewed as the sum of projected individual head outputs O_h :

$$\text{MHSA}(X) = \sum_{h=1}^H O_h W_h^O \quad (32)$$

To mathematically formulate pruning, we introduce a continuous mask variable $z_h \in \{0, 1\}$ for each head:

$$\tilde{O}_h = z_h O_h \quad (33)$$

- When $z_h = 1$, the head is unpruned (dense model).
- When $z_h = 0$, the head is completely pruned.

Estimating Head Importance: Taylor Expansion

The true impact of pruning head h is the exact difference in the loss function:

$$\Delta\mathcal{L}_h = \mathcal{L}(z_h = 0) - \mathcal{L}(z_h = 1) \quad (34)$$

Evaluating this strictly requires a full forward pass for every single head, which is computationally intractable. We bypass this using a **First-Order Taylor Expansion** evaluated around the unpruned state ($z_h = 1$):

$$\mathcal{L}(z_h = 0) \approx \mathcal{L}(z_h = 1) + \left. \frac{\partial \mathcal{L}}{\partial z_h} \right|_{z_h=1} (0 - 1) \quad (35)$$

This simplifies the change in loss to the negative gradient with respect to the mask:

$$\Delta\mathcal{L}_h \approx -\frac{\partial \mathcal{L}}{\partial z_h} \quad (36)$$

Estimating Head Importance: The Chain Rule

How do we compute $\frac{\partial \mathcal{L}}{\partial z_h}$ without explicitly adding z_h to the physical model? We apply the multivariate chain rule.

Because the masked output is $\tilde{O}_h = z_h O_h$, the derivative of the activation with respect to the mask is simply the activation itself:

$$\frac{\partial \tilde{O}_h}{\partial z_h} = O_h \quad (37)$$

Applying the chain rule to the scalar loss function yields the Frobenius inner product between the gradients and the activations:

$$\frac{\partial \mathcal{L}}{\partial z_h} = \sum_{i,j} \frac{\partial \mathcal{L}}{\partial (\tilde{O}_h)_{i,j}} \frac{\partial (\tilde{O}_h)_{i,j}}{\partial z_h} = \nabla_{O_h} \mathcal{L} \cdot O_h \quad (38)$$

Estimating Head Importance: The Final Approximation

Because the loss landscape depends on the input data, we take the expectation of the quantity over a calibration dataset X to find the average gradient-activation product. The importance score $\mathcal{I}(h)$ is defined as the absolute magnitude of this expected change in loss.

Final Importance Score Formulation

$$\mathcal{I}(h) := |\Delta\mathcal{L}_h| = |\mathbb{E}_X [\nabla_{O_h}\mathcal{L} \cdot O_h]| \approx \left| \frac{1}{N} \sum_{i=1}^N \left(\nabla_{O_h^{(i)}}\mathcal{L}^{(i)} \cdot O_h^{(i)} \right) \right| \quad (39)$$

Implementation Efficiency: Both $\nabla_{O_h}\mathcal{L}$ and O_h are naturally computed and stored in memory during a standard backpropagation pass on the calibration set, making this metric extremely cheap to compute globally.

SVD Truncation: The Non-Convex Formulation

Structured pruning via Singular Value Decomposition (SVD) aims to find a low-rank matrix $\hat{W}$ that best approximates the original weight matrix W .

The natural mathematical formulation enforces a strict maximum rank r :

$$\min_{\hat{W}} \|W - \hat{W}\|_F^2 \quad \text{subject to} \quad \text{rank}(\hat{W}) \leq r \quad (40)$$

Eckart-Young-Mirsky Theorem: Although the rank constraint is highly non-convex, the exact global minimum is analytically given by the truncated SVD. If $W = U\Sigma V^T$, then:

$$\hat{W} = \sum_{i=1}^r \sigma_i \mathbf{u}_i \mathbf{v}_i^T \quad (41)$$

where σ_i are the top r largest singular values.

Low-Rank Pruning: The Nuclear Norm

For structured low-rank sparsity, the convex envelope for the rank of a matrix is the

Nuclear Norm: $\|\hat{W}\|_* = \sum_i \sigma_i(\hat{W}) = \text{tr}(\sqrt{\hat{W}^T \hat{W}})$.

In continuous optimization, we can enforce this low-rank structure using two distinct paradigms:

1. The Penalized (Proximal) Formulation

$$\min_{\hat{W}} \mathcal{L}(\hat{W}) + \lambda \|\hat{W}\|_* \quad (42)$$

Solved via Proximal Gradient Descent using **Singular Value Thresholding (SVT)**.

2. The Constrained Formulation

$$\min_{\hat{W}} \mathcal{L}(\hat{W}) \quad \text{subject to} \quad \|\hat{W}\|_* \leq R \quad (43)$$

Solved via **Projected Gradient Descent (PGD)**, enforcing a strict maximum budget R .

SVD Truncation: The Convex Regularization

The regularized, purely convex formulation becomes:

$$\arg \min_{\hat{W}} \frac{1}{2} \|W - \hat{W}\|_F^2 + \lambda \|\hat{W}\|_* \quad (44)$$

Solution via Proximal Operator (Singular Value Thresholding): Instead of hard-truncating to k , we apply soft-thresholding to the singular values:

$$\hat{W}^* = \text{prox}_{\lambda \|\cdot\|_*}(W) = U \text{diag}(\max(\sigma_i - \lambda, 0)) V^T \quad (45)$$

Instead of a hard cutoff at rank r , SVT shrinks all singular values by λ , naturally dropping smaller components to zero.

Solving the Constrained Formulation

The Projection Operator on the nuclear norm ball: $\Pi_{\mathcal{B}_*}(W) = U \text{diag}(\sigma^*) V^T$

The optimal singular values σ^* are simply the projection of the original singular values σ onto the ℓ_1 -ball of radius R :

$$\sigma^* = \arg \min_{\hat{w}} \|\hat{w} - \sigma\|_2^2 \quad \text{subject to} \quad \hat{w} \geq 0, \|\hat{w}\|_1 \leq R \quad (46)$$

This subproblem is solved exactly in $\mathcal{O}(d \log d)$ or $\mathcal{O}(d)$ time using a continuous water-filling or bisection algorithm on the simplex.

Dynamic vs. Fixed Thresholding

Sometimes we may prefer the Constrained approach for MLLM compression

The proximal approach (SVT) shrinks all singular values by a *fixed* hyperparameter λ :

$$\sigma_i^{\text{prox}} = \max(\sigma_i - \lambda, 0) \quad (47)$$

The constrained projection computes a *dynamic* threshold θ to strictly satisfy the constraint:

$$\sigma_i^{\text{proj}} = \max(\sigma_i - \theta, 0) \quad \text{where } \theta \text{ satisfies } \sum_{i=1}^r \max(\sigma_i - \theta, 0) = R \quad (48)$$

The Challenge with SVD in Large Language Models

Despite strong mathematical foundations, standard SVD truncation fails when applied naively to massive LLMs/MLLMs.

Problem 1: Severe Truncation Loss

Because LLM weight matrices are highly dense and encode vast amounts of world knowledge, dropping the trailing singular values completely throws away a massive “tail” of critical information, destroying zero-shot reasoning.

Problem 2: Layer-wise Error Propagation

LLMs are deeply sequential. If we compress early layers (e.g., Layer 2 out of 32), the truncation error compounds exponentially as it passes through the subsequent 30 layers, leading to catastrophic hallucination.

ERC-SVD: Error-Controlled SVD

ERC-SVD (Error-Controlled SVD, 2025) specifically tackles the truncation loss in LLM compression through a two-stage residual compensation strategy.

Instead of purely dropping the tail of the SVD, ERC-SVD extracts useful components from the approximation error itself:

1. **Initial Truncation:** Factorize W to an intermediate rank r_i to get W_{r_i} .
2. **Calculate Residual:** Compute the truncation error matrix: $R = W - W_{r_i}$.
3. **Residual SVD:** Perform a second SVD directly on the residual R to rank r_r , yielding R_{r_r} .
4. **Compensate Residual:** Form the final compressed weight $\hat{W} = W_{r_i} + R_{r_r}$, where $r_i + r_r = r$ is the target rank budget.

Note: When combined with activation-aware weighting (where SVD is applied to WXX^T), this two-stage process captures distinct non-linear activation spaces missed by a single-pass SVD.

ERC-SVD: Partial-Layer Compression

To solve the error propagation problem, ERC-SVD abandons the assumption that all layers should be compressed equally.

The Partial-Layer Strategy

Instead of compressing every layer to 20% sparsity, ERC-SVD keeps the **early layers completely intact** and aggressively compresses the **final layers**.

Why this works:

- Early layers establish the foundational semantic representations and syntactic structure.
- Errors introduced late in the network (e.g., the last 5 layers) do not have enough depth to amplify into catastrophic failure.
- By maintaining a fixed *overall* compression budget, shifting the parameter reduction entirely to the back of the model preserves state-of-the-art accuracy on language modeling and visual reasoning benchmarks.

ERC-SVD as a Bilevel Optimization Problem

We can formulate ERC-SVD as a formal **Bilevel Optimization** problem.

The objective dictates a strict hierarchy: we must first extract the mathematically optimal unweighted structure, and only then optimize the activation-weighted error.

The Bilevel Formulation

$$\min_{W_1, W_2} \frac{1}{2} \|(W - W_1 - W_2)X\|_F^2 \quad (49)$$

Subject to the constraints:

$$\text{rank}(W_2) \leq r_2 \quad (50)$$

$$W_1 \in \arg \min_V \left\{ \frac{1}{2} \|W - V\|_F^2 \quad \text{s.t.} \quad \text{rank}(V) \leq r_1 \right\} \quad (51)$$

Simplifying ERC-SVD: The Truncation Operator

We can simplify this formulation because the lower-level problem has a closed-form analytical solution.

By the Eckart-Young-Mirsky theorem, the inner optimization is exactly the definition of the hard rank- r truncation operator:

$$\mathcal{T}_{r_1}(W) = \arg \min_{\text{rank}(V) \leq r_1} \|W - V\|_F^2 \quad (52)$$

We substitute $W_1 = \mathcal{T}_{r_1}(W)$ and define the **structural residual** as $R = W - \mathcal{T}_{r_1}(W)$ to get a single-level optimization over W_2 :

$$\min_{W_2} \frac{1}{2} \|(R - W_2)X\|_F^2 \quad \text{s.t.} \quad \text{rank}(W_2) \leq r_2 \quad (53)$$

Algorithmically: Extract the unweighted geometry ($W \rightarrow R$), then perform activation-weighted low-rank approximation on the remaining structural error.

ERC-Nuclear Norm: A Convex Capacity Relaxation

We can substitute the non-convex hard rank constraint with the **Nuclear Norm** (the convex envelope of the matrix rank).

The hierarchical logic is identical: we enforce a nuclear capacity budget τ_1 to capture the unweighted structure, and a subsequent budget τ_2 to map the activation tail.

The Nuclear Norm Bilevel Formulation

$$\min_{W_1, W_2} \frac{1}{2} \|(W - W_1 - W_2)X\|_F^2 \quad (54)$$

Subject to the constraints:

$$\|W_2\|_* \leq \tau_2 \quad (55)$$

$$W_1 \in \arg \min_V \left\{ \frac{1}{2} \|W - V\|_F^2 \quad \text{s.t.} \quad \|V\|_* \leq \tau_1 \right\} \quad (56)$$

Simplifying ERC-Nuclear: The Projection Operator

Just as before, the lower-level problem has a known geometric solution: it is the exact Euclidean projection of W onto the nuclear norm ball of radius τ_1 .

Let this projection operator be $\Pi_{\tau_1}(W)$, which applies a dynamic soft-thresholding shift to the singular values of W . We substitute $W_1 = \Pi_{\tau_1}(W)$.

We define the **nuclear residual** $R_* = W - \Pi_{\tau_1}(W)$. This residual explicitly contains the “excess” magnitude of the heavy tail shaved off during projection.

The Collapsed ERC-Nuclear Formulation

$$\min_{W_2} \frac{1}{2} \|(R_* - W_2)X\|_F^2 \quad \text{s.t.} \quad \|W_2\|_* \leq \tau_2 \quad (57)$$

Interpretation: The first stage gently shaves down the heavy tail of the spectrum. The second stage applies activation weighting strictly to recover that shaved mass.

Context Compression: Token Merging

The Sequence Length Bottleneck

As we process high-resolution images or videos in MLLMs, the number of visual tokens N grows massively.

The Attention Complexity Wall

The time and memory complexity of the self-attention mechanism scales quadratically with sequence length: $\mathcal{O}(N^2d)$.

- A 1024×1024 image divided into 14×14 patches generates over 5,300 visual tokens.
- Video inputs push this to 100,000+ tokens, completely dominating the KV cache.

The Opportunity: Visual data is inherently highly redundant (e.g., large patches of solid sky or blank walls). We do not need to preserve every token.

What is Token Merging?

Instead of permanently altering the model weights (Pruning) or their precision (Quantization), **Token Merging (ToMe)** dynamically reduces the computation payload during inference.

- **Mechanism:** Identifies spatially or semantically redundant tokens and fuses them together into a single representation before they pass through the self-attention layer.
- **Outcome:** The sequence length shrinks monotonically as data flows deeper into the network ($N \rightarrow N - r \rightarrow N - 2r \dots$).
- **Advantage:** Requires zero retraining. It is an inference-time intervention that dynamically accelerates both the Vision Encoder and the LLM Backbone.

Relationship to Quantization: The Hardware Synergy

Combining both yields multiplicative efficiency gains for MLLMs.

The Synergy in Prefill vs Decoding

- **Prefill Phase:** Dominated by processing thousands of visual tokens at once. This is strictly **Compute-Bound** ($\mathcal{O}(N^2)$). Token Merging slashes this prefill latency by dropping N .
- **Decoding Phase:** Generating text token-by-token. This is strictly **Memory-Bandwidth Bound**. Quantization (e.g., W4A16) accelerates this by shrinking weight load times.

To deploy a 70B MLLM for real-time video QA, we need both Token Merging for the visual context, and Quantization for the autoregressive generation.

The Three Axes of Compression

Quantization, Token Merging, and Pruning do not compete—they each attack a fundamentally different variable in the hardware bottleneck equation.

Recall the computational cost of a Transformer layer:

$$\text{FLOPs} \approx \underbrace{4\mathbf{N}\mathbf{d}^2}_{\text{Linear Projections}} + \underbrace{2\mathbf{N}^2\mathbf{d}}_{\text{Self-Attention}} \quad (58)$$

- **Token Merging (Data Axis):** Shrinks $\mathbf{N}$. Brutally effective at cutting the $\mathcal{O}(N^2)$ attention compute and KV cache size during visual prefill.
- **Pruning (Structural Axis):** Shrinks $\mathbf{d}$ (width) or depth. Because linear projections scale by $\mathbf{d}^2$, structured pruning directly slashes the baseline arithmetic workload (FLOPs) required per token.
- **Quantization (Precision Axis):** Shrinks the bit-width. It doesn't change the number of operations, but drastically speeds up the memory-bandwidth bottleneck of loading the $\mathbf{d} \times \mathbf{d}$ parameter matrices from VRAM.

A Popular Strategy: Token Merging (ToMe) and Variants

Some References

1. Bolya et al., (2023). *"Token Merging: Your ViT But Faster."* (ICLR).
2. Yang et al., (2025). *TopV: Compatible Token Pruning with Inference Time Optimization for Fast and Low-Memory Multimodal Vision Language Model* (CVPR)
3. Shao et al., (2025). *HoliTom : Holistic Token Merging for Fast Video Large Language Models* (NeurIPS)

Core Concept: Instead of dropping tokens entirely, we can fuse them. If two tokens are highly similar, they are averaged together. To do this efficiently, ToMe avoids the $\mathcal{O}(N^2)$ cost of computing pairwise similarities between all tokens.

ToMe: Bipartite Soft Matching

ToMe executes token merging via a fast **Bipartite Matching** algorithm.

Given a set of N tokens at a specific layer:

1. **Partition:** Randomly or heuristically split the tokens into two disjoint sets: $\mathcal{A}$ and $\mathcal{B}$, each of size $N/2$.
2. **Similarity:** Compute the cosine similarity only between tokens in $\mathcal{A}$ and tokens in $\mathcal{B}$ (using the Attention keys $\mathbf{k}_i$).
3. **Bipartite Edge:** For each token in $\mathcal{A}$, find its single most similar token in $\mathcal{B}$.
4. **Merge:** Select the top r most similar pairs globally, and merge them by taking their average feature vector.

By splitting into bipartite sets, the matching complexity drops to $\mathcal{O}(N^2/4)$, and avoids complex clustering overhead.

ToMe: Tracking Token Proportions

When multiple tokens merge, the resulting fused token represents a larger physical area of the image. The model must be aware of this to maintain proper attention weights.

ToMe introduces a **size tracking vector** $\mathbf{s} \in \mathbb{R}^N$ initialized to 1. When token i and j merge, the new token size is $s_i + s_j$.

Proportional Attention Adjustment: We modify the self-attention softmax to account for the mass of merged tokens:

$$A_{ij} = \text{softmax} \left(\frac{\mathbf{q}_i^T \mathbf{k}_j}{\sqrt{d}} + \log(s_j) \right) \quad (59)$$

Adding $\log(s_j)$ ensures that heavily merged background tokens (which represent many original patches) still receive the statistically correct aggregate attention probability from the queries.

Conclusion

Summary: The Approximate Computing Toolkit

The Trinity of MLLM Compression

Deploying massive MLLMs (70B+) for real-time multimodal applications requires a synergistic approach across three distinct axes:

- **Quantization (The Precision Axis):** Methods like AWQ, GPTQ, and TurboQuant target the memory bandwidth wall. By compressing weights and KV caches to 4-bit (or lower), we fit massive models onto edge GPUs while preserving mathematical fidelity.
- **Pruning (The Structural Axis):** Techniques from $S + L$ Decomposition to Taylor-based ViT Pruning permanently remove redundant parameters (d), fundamentally lowering the FLOPs required per token.
- **Token Merging (The Data Axis):** Methods based on ToMe dynamically shrink the input sequence length (N), breaking the $\mathcal{O}(N^2)$ compute bottleneck of long visual contexts without retraining.

Thank You!

Questions?

Prof. Sathya N. Ravi
University of Illinois Chicago

`sathya-uic.github.io`